

**EXERCISE 9.11**

Can you think of any other primitives that might be useful for letting the programmer specify what error(s) in an `or` chain get reported?

---

Note that we still haven't written an implementation of our algebra! But this exercise has been more about making sure our combinators provide a way for users of our library to convey the right information to the implementation. It's up to the implementation to figure out how to use this information in a way that satisfies the laws we've stipulated.

## 9.6 Implementing the algebra

By this point, we've fleshed out our algebra and defined a `Parser[JSON]` in terms of it.<sup>14</sup> Aren't you curious to try running it?

Let's again recall our set of primitives:

- `string(s)`—Recognizes and returns a single `String`
- `regex(s)`—Recognizes a regular expression `s`
- `slice(p)`—Returns the portion of input inspected by `p` if successful
- `label(e)(p)`—In the event of failure, replaces the assigned message with `e`
- `scope(e)(p)`—In the event of failure, adds `e` to the error stack returned by `p`
- `flatMap(p)(f)`—Runs a parser, and then uses its result to select a second parser to run in sequence
- `attempt(p)`—Delays committing to `p` until after it succeeds
- `or(p1, p2)`—Chooses between two parsers, first attempting `p1`, and then `p2` if `p1` fails in an uncommitted state on the input

**EXERCISE 9.12**

*Hard:* In the next section, we'll work through a representation for `Parser` and implement the `Parsers` interface using this representation. But before we do that, try to come up with some ideas on your own. This is a very open-ended design task, but the algebra we've designed places strong constraints on possible representations. You should be able to come up with a simple, purely functional representation of `Parser` that can be used to implement the `Parsers` interface.<sup>15</sup>

---

<sup>14</sup> You may want to revisit your parser to make use of some of the error-reporting combinators we just discussed in the previous section.

<sup>15</sup> Note that if you try running your JSON parser once you have an implementation of `Parsers`, you may get a stack overflow error. See the end of the next section for a discussion of this.